Java Programming

CPT111 – Lab 3

Teng Ma

Xi'an Jiaotong-Liverpool University
西交利物浦大学

CPT111 Java Programming

Lab 3

Control Flow

Lab Exercise #3.1

- TAs will lead you to **test**, **debug**, and **solve** a control flow problem

QUIZ

Lab Exercise #3.1 Armstrong Number

- The TA explains the problem to students:

An Armstrong number is an *n*-digit number that equals the sum of the *n*th power of its digits.

For example,

- 153 is a three-digit number where the sum of the cubes of the individual digits equal 153
- 1634 is a four-digit number where the sum of the 4th power of the individual digits equal 1634

$$153 = (1)^3 + (5)^3 + (3)^3$$
$$1634 = (1)^4 + (6)^4 + (3)^4 + (4)^4$$

Write a Java program *Armstrong* that on input two positive integers *a* and *k*, prints the first *k* Armstrong numbers greater than or equal to *a*.

Lab Exercise #3.1 – Activity 1: Testing

- Two test cases are **given** in Lab Exercise #3.1
  - include them in your code
  - however, they are not enough
    to show that your code is correct...

- The TA will suggest more test cases:
  - Armstrong number of smaller number of digits?
  - Armstrong number of larger number of digits?

Test case 1:
Input:
100
2
Output:
153
370

Test case 2:
Input:
1500
1
Output:
1634

Lab Exercise #3.1 – Activity 2: Decomposition

- TA will break down the steps to solve this problem

- One possible steps are:
  - get the input numbers from the user
  - start with the given number
  - determine how many digits are in the number
  - compute the sum of its digits, each raised to the power corresponding to the number of digits
  - verify whether the number is an Armstrong number
  - repeat for next number, while keeping track of how many Armstrong numbers are found
  - ...

solve each subproblem

Lab Exercise #3.1 – Activity 3: Debugging and Visualizing

- It is a good practice to test and debug each steps independently
- The TA will show using **debugger** and **visualizer** that this code will not work on some input
  - for example:

```
// Find how many digits is in a, to be exponent n
int n = 0;
int b = a;
while (b != 1) {  // the correct code is while (b != 0) {
    b = b / 10;
    n++;
}
```

Lab Exercise #3.1 – Activity 4: Solving

- The TA will explain a solution to the Armstrong Number problem

- Alternative activity:
  - Use XIPU AI to help you with problem solving:
    English: https://www.learningmall.cn/en/ceie/computational-thinking-and-pair-programming-with-xipu-ai.html
    Chinese: https://www.learningmall.cn/zh/ceie/computational-thinking-and-pair-programming-with-xipu-ai.html

Lab 3 Challenge – Hailstone Sequence Length

- You will practice debugging, visualizing and solving this LM Quiz, and show your work to your lab TAs to earn marks

Lab 3 Challenge - Hailstone Sequence Length

- There are 3 tasks to complete

The *Hailstone sequence* is computed as follows:

- Start with an integer number n
- If n is even, the next number in the sequence is n/2
- If n is odd, the next number is 3n+1
- Continue until it reaches 1

For example,
for n = 3, the sequence is 3, 10, 5, 16, 8, 4, 2, 1
and for n = 12, the sequence is 12, 6, 3, 10, 5, 16, 8, 4, 2, 1

Write a Java program *HailStone* that on input a positive integer n, prints the ***length*** of the Hailstone sequence starting at n.

Lab 3 Challenge – Task 1: Create Test Cases

- You are provided with 2 given test cases

For example,
for n = 3, the sequence is 3, 10, 5, 16, 8, 4, 2, 1
and for n = 12, the sequence is 12, 6, 3, 10, 5, 16, 8, 4, 2, 1

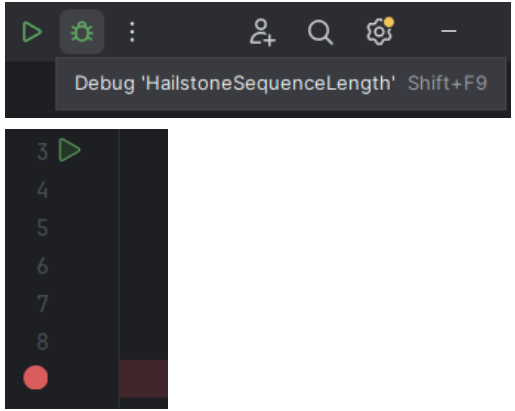
Test case 1:
Input:
3
Output:
8

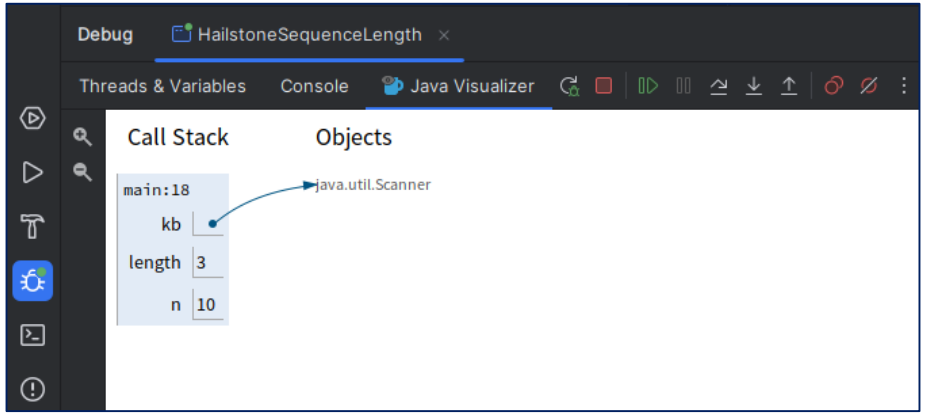
Test case 2:
Input:
12
Output:
10

- Create **at least two more** test cases, and tell your TA why you create them
  - you need to give **good reasoning** to earn a full mark in this task
  - a short hailstone sequence?
  - a very short hailstone sequence?
  - special case of a hailstone sequence that always goes smaller?
  - a very long hailstone sequence on a small starting number? Ask XIPU AI!

Lab 3 Challenge – Task 2: Visualize Your Code

- Write an initial code that possibly contains one or more bugs
- Run the **debugger** and **visualizer**, (check Lecture 2)





- Explain to your TA **why** you suspect that there is a bug in that line
  - infinite loop?
  - off-by-one bug?
  - logical error resulting in incorrect length output?

Lab 3 Challenge – Task 3: Solve the Problem

- Solve the Hailstone Sequence Length problem
- Show to your TA that you have passed the LM Quiz

	Input	Expected	Got	
✔	3	8	8	✔
✔	12	10	10	✔

Passed all tests! ✔

Correct

Marks for this submission: 100.00/100.00.

there are additional hidden tests

- Briefly explain to the TA you solution!

Thank you for your effort!

- If you finish early, feel free to use the remaining time to work on other online exercises
  - use this opportunity to practice your new skills in testing, debugging, and visualizing your code!

- You may leave the lab once you have complete the Lab 3 Challenge and it has been ASSESSED by a TA!
  - Please note that the **Lab 3 Challenge** is **graded** and contributes to your CW2 marks